

Relazione Progetto SABD A.A. 2025/2026

Progetto 1

Simone Ercole

Matricola: 0360105

Dipartimento di Ingegneria

Università degli Studi di Roma "Tor Vergata"

Corso di Laurea Magistrale in Ingegneria Informatica

Federico Pepe

Matricola: 0365363

Dipartimento di Ingegneria

Università degli Studi di Roma "Tor Vergata"

Corso di Laurea Magistrale in Ingegneria Informatica

Abstract—Il presente elaborato descrive l'architettura e l'implementazione di un sistema distribuito finalizzato all'analisi di grandi volumi di dati, nello specifico riguardanti i voli aerei forniti dal Dipartimento dei Trasporti degli Stati Uniti (DOT). La pipeline di data ingestion, ingegnerizzata tramite Apache NiFi, provvede al trasferimento dei dati verso un Data Lake centralizzato su HDFS. L'elaborazione delle query e l'analisi computazionale sono affidate ad Apache Spark, mentre l'intera infrastruttura è containerizzata mediante Docker Compose e orchestrata in modo automatizzato attraverso il framework Apache Airflow.

I. INTRODUZIONE

Nel panorama tecnologico contemporaneo, il dato ha assunto una centralità strategica, rendendo la capacità di estrazione di conoscenza un'attività determinante per il vantaggio competitivo. In tale ambito, l'analisi dei flussi del traffico aereo risulta essenziale per identificare i fattori determinanti dei ritardi, i quali rappresentano un onere economico rilevante per le compagnie di settore. Il presente lavoro illustra l'impiego del framework Apache Spark — piattaforma d'eccellenza per l'elaborazione distribuita — nell'analisi di un dataset storico relativo al primo quadrimestre del 2025 negli Stati Uniti d'America.

L'indagine è volta alla risoluzione di specifiche query analitiche finalizzate a quantificare i ritardi (sia in fase di partenza che di arrivo), l'incidenza delle cancellazioni e le principali concause dei disservizi, al fine di delinearne i fattori scatenanti. L'elaborato descrive nel dettaglio l'architettura di sistema implementata, le scelte tecnologiche adottate e l'intero processo di data ingestion. Infine, vengono presentati i risultati ottenuti, integrati da una discussione critica e da una valutazione dell'efficacia e delle performance delle query implementate.

II. DATASET E REQUISITI

Il dataset in esame comprende circa 2.200.000 record, ciascuno dei quali rappresenta un evento di volo caratterizzato dai seguenti attributi:

- **Vettore aereo:** compagnia responsabile del servizio;
- **Riferimento temporale:** data del volo (anno, mese e giorno);
- **Tratta:** aeroporti di origine e di destinazione;
- **Programmazione oraria:** orario di partenza schedato;

- **Metriche di scostamento:** ritardo effettivo in fase di partenza e di arrivo;
- **Stato del volo:** indicatore della condizione operativa (es. regolare, cancellato, deviato);
- **Cause dei disservizi:** tassonomia delle cause che hanno originato il ritardo.

Ai fini del progetto, l'analisi si focalizza in particolare sui seguenti indicatori:

- **Analisi dei ritardi:** valutazione degli scostamenti temporali in partenza e in arrivo;
- **Tasso di cancellazione:** definito come il rapporto percentuale tra i voli cancellati e la totalità dei voli pianificati;
- **Determinanti delle cancellazioni:** analisi qualitativa e quantitativa delle cause scatenanti, essenziale per l'identificazione dei pattern di inefficienza.

III. ARCHITETTURA DEL SISTEMA

L'architettura del sistema si articola in molteplici servizi, tutti orchestrati localmente attraverso Docker Compose. Nello specifico, l'infrastruttura comprende:

- **Apache NiFi:** Impiegato per la gestione e il coordinamento dell'intera pipeline di *Data Ingestion*.
- **HDFS:** Funge da *Data Lake* per l'archiviazione dei dati grezzi nei formati Avro e Parquet. L'architettura è configurata con un NameNode e un DataNode.
- **Apache Spark:** Il framework di riferimento per l'elaborazione distribuita dei dati. È configurato come cluster *standalone*, prevedendo un nodo Master e diverse configurazioni per i nodi Worker.
- **Redis:** Sistema di *storage in-memory* utilizzato per memorizzare i risultati delle elaborazioni in output da Spark, fungendo da livello di *servicing* ad alte prestazioni per Grafana.
- **Grafana:** Piattaforma dedicata alla *Data Visualization*, utilizzata per la creazione di dashboard interattive che consentono un'esplorazione e una visualizzazione ottimale dei risultati delle query.

Al fine di garantire una comunicazione sicura e isolata, tutti i *container* sono interconnessi tramite una rete Docker dedicata, denominata *HDFS_Network*.

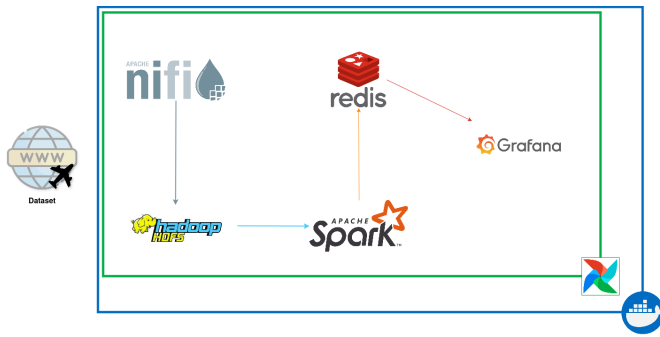


Fig. 1. Schema di comunicazione orchestrato.

A. Data ingestion

La fase di data ingestion è stata implementata ingegnerizzando un flusso di lavoro all'interno del framework Apache NiFi. Nello specifico, la pipeline orchestra dinamicamente il download dei file in formato CSV e la loro successiva conversione nello standard Avro. Tale scelta architetturale è motivata dalle modalità native di processamento di NiFi, il quale opera attraverso una logica row-oriented. Di conseguenza, una conversione diretta nel formato Parquet avrebbe introdotto un overhead computazionale non trascurabile, costringendo il framework a mantenere l'intero dataset in memoria volatile prima di poter eseguire la serializzazione. L'approccio adottato mitiga questo vincolo, garantendo scalabilità e prestazioni ottimali all'aumentare della dimensione dei dati. Una volta completata la pipeline di ingestione, i record vengono persistiti all'interno del file system distribuito HDFS, secondo una logica di partizionamento basata sul mese di riferimento. Di seguito vengono dettagliati i processori e i connettori specifici impiegati nella pipeline:

- **InvokeHTTP**: il processore effettua una richiesta HTTP di tipo GET verso l'URL specificato, finalizzata al download del dataset in formato compresso *.tar.gz*.
- **CompressContent**: deputato all'esecuzione della prima fase di decompressione, isolando l'archivio dall'estensione *.gz*.
- **UnpackContent**: esegue la decompressione del file *.tar*, estraendo i quattro file in formato *.csv* corrispondenti ai dataset mensili.
- **UpdateAttribute**: impiegato per aggiornare i metadati dei FlowFile, nello specifico modificando l'attributo relativo al nome del file e predisponendo l'estensione *.avro* per le fasi successive.
- **QueryRecord**: componente chiave per il filtraggio e la proiezione dei dati, configurato per selezionare esclusivamente le colonne pertinenti alle interrogazioni analitiche successive. Il processore si avvale di due *Controller Service* per la serializzazione:
 - **CSVReader**: analizza il flusso in ingresso, inferendo e interpretando lo schema nativo del file.
 - **AvroRecordSetWriter**: serializza i record filtrati in formato *.avro*, preservando la coerenza dello schema strutturale.

La selezione degli attributi d'interesse avviene mediante la query SQL interna *output_avro*. Tale operazione di *data pruning* consente di scartare preventivamente le colonne non necessarie, ottimizzando l'occupazione di memoria e l'efficienza delle computazioni a valle.

- **PutHDFS**: modulo terminale della pipeline, responsabile della persistenza dei quattro file in formato *.avro*. Il componente scrive i dati sul file system distribuito Hadoop (HDFS) instradandoli verso i path di destinazione specifici, precedentemente definiti in base alla logica di partizionamento mensile.

B. Preprocessamento

Il processo ha inizio con l'acquisizione dei file grezzi in formato *Avro*, i quali vengono convertiti nel formato *Parquet* al fine di ottimizzare le prestazioni nelle successive fasi di elaborazione.

Successivamente, si è proceduto all'eliminazione dei record privi di attributi fondamentali, quali l'identificativo della compagnia aerea, il riferimento temporale e lo stato operativo del volo. Successivamente, sono stati implementati dei filtri di coerenza logica per distinguere i valori nulli "fisiologici" dai dati corrotti. Nello specifico, i record privi di informazioni sul ritardo in partenza (*DEP_DELAY*) o in arrivo (*ARR_DELAY*) sono stati mantenuti nel *dataset* esclusivamente se tale assenza risultava giustificata dalla cancellazione o dal dirottamento del volo. Questo ha permesso di preservare integralmente le dinamiche dei disservizi reali (come le cancellazioni avvenute in pista dopo il *pushback*), scartando unicamente le anomalie di sistema.

Infine, si è effettuata la normalizzazione per le cause di ritardo (es. meteo, vettore, sicurezza), decidendo di forzare l'assenza di dati a zero. Questa condizione si verifica intrinsecamente per i voli non giunti a destinazione, oppure per i voli che hanno registrato un ritardo all'arrivo inferiore ai 15 minuti [1]; una soglia sotto la quale, in conformità con le direttive aeronautiche statunitensi (FAA), il volo non è classificato come ufficialmente in ritardo.

C. Implementazione ed Elaborazione delle Query

Le query richieste sono state implementate in script PySpark indipendenti, strutturati sfruttando sia le *API DataFrame* sia le *API RDD* di **Apache Spark**. Tutti gli script effettuano la lettura dei dati in formato *Parquet* partizionato in *HDFS*.

Allo stesso modo, i risultati delle elaborazioni vengono salvati nuovamente su *HDFS* in formato *CSV*.

D. Coordinamento tramite Apache Airflow

L'orchestrazione dell'intera *pipeline* dei dati, a partire dalla fase di *Data Ingestion* fino all'esportazione dei risultati, è stata affidata ad **Apache Airflow**. Sono stati implementati complessivamente 5 *DAG*.

Nello specifico, la logica di schedulazione è così strutturata:

- **Data Ingestion e Preprocessing**: Un primo DAG è interamente dedicato all'acquisizione e alla pulizia preliminare dei dati. Al suo interno è stato configurato un

operatore `HdfsSensor`, il cui compito è mantenere il *workflow* in attesa finché i quattro *processor* finali di *Apache NiFi* non generano un apposito file `_SUCCESS`. La presenza di questo file funge da *flag* di sincronizzazione, garantendo che tutti i dati in formato *Avro* siano stati scritti correttamente e risultino pienamente disponibili all'interno del file system distribuito.

- **Esecuzione delle Query:** Sono stati definiti altri 3 DAG paralleli e indipendenti, ciascuno ingegnerizzato per avviare ed eseguire una delle tre *query* analitiche richieste dal progetto.
- **Esportazione Dei risultati:** L'ultimo DAG è utilizzato per esportare i dati da HDFS a Redis.

E. Esportazione dei Risultati

I risultati in formato CSV prodotti da ciascuna query e salvati su HDFS vengono successivamente letti da un apposito script PySpark (`export_to_redis.py`) e caricati su un'istanza *Redis*. Per ogni query, i dati vengono memorizzati in forma grezza a granularità singola (ad esempio, utilizzando una chiave del tipo `q1:AA:1`). Per ottimizzare l'accesso a tali record, viene impiegato il modulo *Redisearch* per la creazione di indici dedicati; questo permette di eseguire query iterative e flessibili sfruttando l'efficienza degli *indici inversi*.

F. Visualizzazione dei Risultati

La fase di presentazione e analisi visiva dei dati è stata implementata tramite *Grafana*. La piattaforma ha consentito la progettazione di dashboard interattive mirate alla generazione e alla visualizzazione dei grafici, basandosi sui risultati finali elaborati dalle query.

IV. QUERY

Query 1: Facendo riferimento al dataset dei voli relativi alle compagnie aeree AA (*American Airlines*) e DL (*Delta*), calcolare la media, il massimo ed il minimo del ritardo in partenza (`DEP_DELAY`) ed il rate di cancellazione dei voli (`CANCELLATION_RATE`) per ciascun mese. Nell'analisi del ritardo in partenza si considerino esclusivamente i voli non cancellati.

Approccio RDD Standard (oggetti Row): Segue un paradigma MapReduce esplicito e manuale (tramite `map` e `reduceByKey`), operando direttamente sugli oggetti Row estratti dal DataFrame. Sebbene sia più intuitivo per l'accesso ai dati, comporta un maggiore overhead computazionale dovuto alla serializzazione e deserializzazione continua degli oggetti PySpark in Python.

Approccio RDD Ottimizzato (Tuple e Indici Dinamici): Mantiene il paradigma MapReduce distribuito, ma converte preventivamente gli oggetti Row in Tuple native Python. Sfruttando la risoluzione dinamica degli indici delle colonne a livello di driver, riduce drasticamente l'overhead di accesso ai dati, garantendo tempi di esecuzione nettamente inferiori rispetto all'approccio RDD standard.

Approccio DataFrame: Sfrutta l'API dichiarativa di Spark SQL (`groupBy` e `agg`) e delega l'ottimizzazione al *Catalyst*

optimizer, garantendo un codice più conciso, efficiente e diretto.

Di seguito si riportano i risultati ottenuti:

TABLE I
RISULTATI DELLA QUERY 1

Mese	Airline	Media	Minimo	Massimo	Canc_Rate
1	AA	12.89	-34.0	3298.0	3.73
1	DL	12.83	-30.0	1209.0	2.80
2	AA	13.28	-27.0	3403.0	1.32
2	DL	11.35	-30.0	1222.0	0.23
3	AA	15.85	-27.0	3288.0	1.52
3	DL	10.32	-32.0	1176.0	0.49
4	AA	15.96	-28.0	2595.0	1.43
4	DL	8.71	-30.0	1145.0	0.51

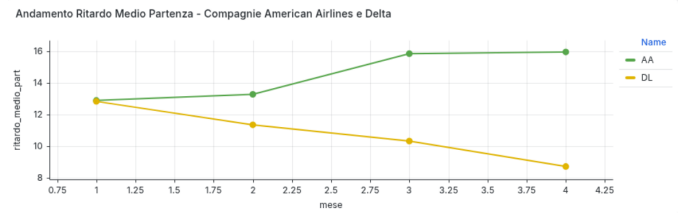


Fig. 2. Andamento Ritardo Medio Per American Airline e Delta

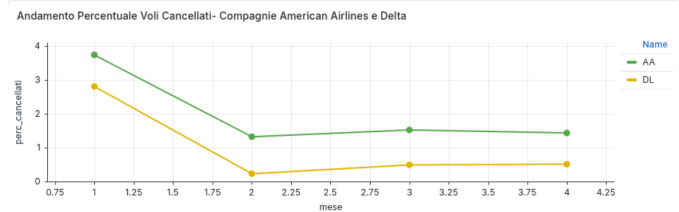


Fig. 3. Andamento Percentuale Cancellazione Per American Airline e Delta

Query 2: Per ciascuna compagnia (`OP_UNIQUE_CARRIER`), calcolare il numero totale di voli non cancellati (`CANCELLED`) e non deviati (`DIVERTED`), il valore medio di ritardo in arrivo (`ARR_DELAY`) ed il valore medio delle componenti di ritardo:

- `CARRIER_DELAY`
- `WEATHER_DELAY`
- `NAS_DELAY`
- `SECURITY_DELAY`
- `LATE_AIRCRAFT_DELAY`

Si considerino solo le compagnie con almeno 500 voli. Infine calcolare la classifica delle prime dieci compagnie ordinate per ritardo medio d'arrivo decrescente.

Approccio RDD Standard (oggetti Row): Segue un paradigma MapReduce esplicito e manuale (tramite `map` e `reduceByKey`), operando direttamente sugli oggetti Row estratti dal DataFrame.

Approccio RDD Ottimizzato (Tuple e Indici Dinamici): Mantiene il paradigma MapReduce distribuito, ma converte preventivamente gli oggetti Row in Tuple native Python.

Approccio DataFrame: Sfrutta l'API dichiarativa di Spark SQL (`groupBy` e `agg`) e delega l'ottimizzazione al *Catalyst optimizer*, garantendo un codice più conciso, efficiente e diretto.

Di seguito si riportano i risultati ottenuti:

TABLE II
RISULTATI DELLA QUERY 2

Airline	N°Voli	Totale	Carrier	Weather	Nas	Security	L.A
OH	78476	17.16	7.24	1.66	3.31	0.03	12.13
F9	65965	11.32	5.92	0.49	3.95	0.0	10.42
G4	42596	9.54	5.68	2.04	4.27	0.16	7.54
AA	302790	8.98	5.95	1.01	2.97	0.02	7.41
OO	259692	7.59	7.93	2.47	2.68	0.02	3.38
B6	75680	6.14	4.49	0.43	4.31	0.03	7.08
HA	25707	5.81	4.46	0.54	0.1	0.02	3.29
MQ	87416	5.32	3.09	1.55	3.06	0.02	5.15
DL	312503	4.19	5.87	0.89	2.79	0.01	4.06
UA	248433	2.46	3.39	0.61	3.83	0.0	4.48

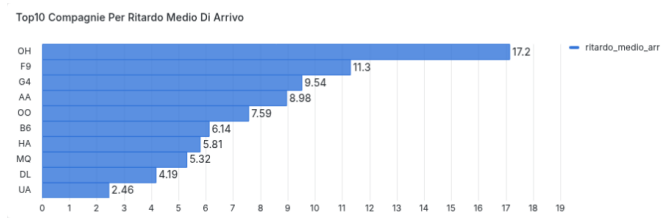


Fig. 4. Top10 compagnie per ritardo medio di arrivo

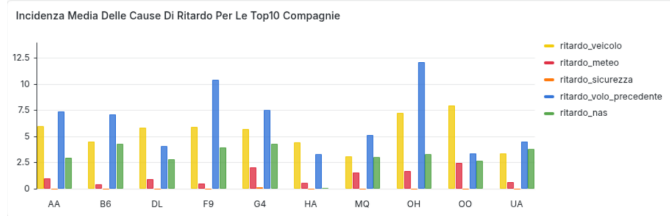


Fig. 5. Incidenza delle cause di ritardo

Query 3: Facendo riferimento al dataset dei voli relativi alle compagnie aeree AA (American Airlines), DL (Delta), UA (United), e WN (Southwest), aggregare i dati di ciascuna compagnia sulle 24 ore della giornata. L'ora del giorno deve essere ricavata a partire dal campo `CRS_DEP_TIME`, considerando le 24 fasce orarie giornaliere. Per ciascuna compagnia e per ciascuna fascia oraria, considerando i valori di `DEP_DELAY` dei voli non cancellati calcolare:

- 25° Percentile
- 50° Percentile (Mediana)
- 75° Percentile
- 90° Percentile

Inoltre, per ogni compagnia, calcolare il valore minimo e massimo del ritardo in partenza `DEP_DELAY` sull'intero dataset.

Approccio RDD Standard (Oggetti Row): Segue un paradigma MapReduce esplicito e manuale, implementato

tramite le trasformazioni `map` e `reduceByKey`, operando direttamente sugli oggetti `Row` nativi del `DataFrame`. Per il calcolo dei percentili, questo approccio si avvale dell'algoritmo *t-digest*, che stima i quantili raggruppando i dati in centroidi e applicando un'interpolazione lineare, producendo così risultati con valori decimali ad alta precisione.

Approccio RDD Ottimizzato (Tuple): Condividendo la medesima infrastruttura di calcolo dell'approccio standard, adotta anch'esso l'algoritmo *t-digest* per i percentili, restituendo i medesimi valori decimali interpolati ma elaborandoli con un'efficienza computazionale nettamente superiore.

Approccio DataFrame: Per la stima dei percentili, utilizza la funzione `percentile_approx` basata sull'algoritmo di *Greenwald-Khanna*. A differenza del *t-digest*, questo metodo opera mantenendo in memoria un sottoinsieme compresso di osservazioni reali: restituendo valori campionati direttamente dal dataset originale, produce risultati interi (o con parte decimale nulla) coerenti con il tipo di dato di partenza.

Di seguito si riportano i risultati ottenuti:

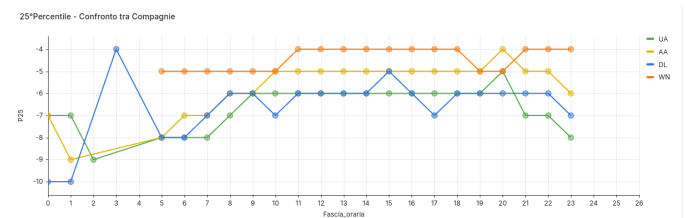


Fig. 6. 25° Percentile confrontato per compagnia

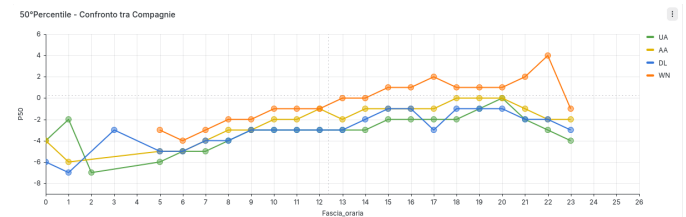


Fig. 7. 50° Percentile confrontato per compagnia

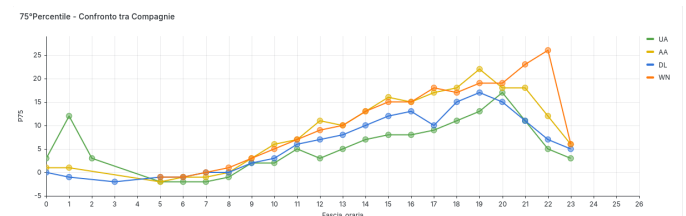


Fig. 8. 75° Percentile confrontato per compagnia

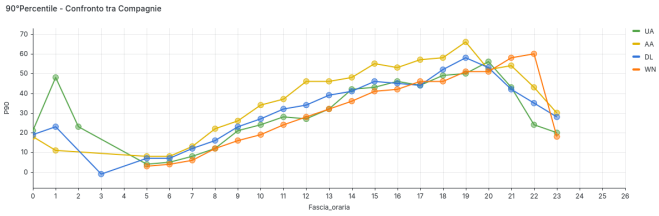


Fig. 9. 90° Percentile confrontato per compagnia

TABLE III
RISULTATI DELLA QUERY 3

Airline	Minimo	Massimo
AA	-34.0	3403.0
DL	-32.0	1222.0
UA	-50.0	1349.0
WN	-24.0	721.0

V. ANALISI DELLE PRESTAZIONI

È stata condotta una valutazione sperimentale dei tempi di processamento delle query per analizzare l'impatto del parallelismo e confrontare l'efficienza delle diverse API offerte da Apache Spark: l'API DataFrame e RDD.

A. Metodologia

- 1) **Configurazione dell'ambiente Spark:** Per garantire l'indipendenza delle misurazioni, per ogni esecuzione è stata istanziata una nuova `SparkSession`. I parametri di risorsa assegnati ai worker sono stati mantenuti costanti come segue:

- **Core:** Fissato a 2 core per ciascun nodo worker.
- **Memoria RAM:** Fissata a 2 GB per nodo.

Durante l'intera fase sperimentale, l'unica variabile di dimensionamento modificata è stato il numero di **Executor** attivi, incrementato progressivamente da 1 fino a un massimo di 3.

- 2) **Replicazione degli esperimenti:** Ciascuna combinazione di Query e configurazione del cluster è stata eseguita 10 volte consecutive. Nel calcolo delle metriche aggregate si è scelto deliberatamente di mantenere la prima esecuzione di *warm-up*. Sebbene questa iterazione introduca una marcata variabilità nel campione dovuta ai tempi fisiologici di inizializzazione di Spark (*Cold Start*), la sua inclusione permette di ottenere un tempo medio che modella fedelmente il costo di un tipico scenario di *batch processing* (in cui un job isolato viene sottomesso, allocato e terminato, subendo interamente l'overhead di avvio). La ripetizione per le restanti 9 iterazioni a regime fa sì che la stima finale ammortizzi il transitorio iniziale, offrendo una valutazione complessiva robusta e rappresentativa.
- 3) **Criteri di misurazione:** I tempi operativi sono stati rilevati tramite la funzione `time.time()` (libreria standard Python), registrando i *timestamp* immediatamente

prima della lettura dei dati Parquet da HDFS e subito dopo l'operazione di scrittura finale. Al fine di isolare i tempi di computazione puri ed evitare colli di bottiglia legati all'I/O su disco, per le prime nove iterazioni la scrittura è stata simulata sfruttando il formato di output `"noop"`. La materializzazione effettiva dei risultati su HDFS in formato CSV è avvenuta unicamente nel corso della decima e ultima iterazione.

- 4) **Aggregazione e Analisi Statistica dei Risultati:** Al termine del ciclo di misurazioni per ciascuna configurazione, i dati raccolti sull'intero set delle 10 iterazioni sono stati sottoposti ad analisi statistica al fine di valutarne l'affidabilità e la dispersione. Nello specifico, per ogni esperimento sono state calcolate le seguenti metriche:

- **Media campionaria ($\bar{X}$):** Il tempo medio di esecuzione, utilizzato come metrica di riferimento principale per la valutazione delle prestazioni e per la rappresentazione tabellare dei risultati.
- **Deviazione standard campionaria (s):** Calcolata con $n - 1$ gradi di libertà per quantificare la dispersione dei tempi rispetto alla media. A causa della deliberata inclusione della prima iterazione di *warm-up*, questo valore riflette la forte escursione temporale tra l'inizializzazione dell'ambiente e le successive esecuzioni a regime (un fenomeno particolarmente evidente nell'approccio DataFrame).
- **Intervallo di confidenza al 95%:** Stimato per valutare la precisione statistica della media calcolata sotto un livello di significatività $\alpha = 0.05$. Considerando la dimensione ridotta del campione ($n = 10$), si è adottata la distribuzione *t di Student* con 9 gradi di libertà ($n - 1$), definendo il margine di errore come:

$$ME = t_{\alpha/2, n-1} \times \frac{s}{\sqrt{n}} \quad (1)$$

dove il valore critico bilaterale tabulato $t_{0.025, 9}$ è pari a 2.262. Si evidenzia che la marcata varianza indotta dall'asimmetria positiva del *warm-up* produce talvolta un limite inferiore dell'intervallo matematicamente negativo; in tali circostanze, non potendo il tempo fisico assumere valori minori di zero, l'estremo inferiore dell'intervallo viene formalmente troncato a zero, evidenziando la natura non strettamente normale della distribuzione campionaria iniziale in Spark.

B. Risultati

TABLE IV
RISULTATI QUERY 1

Query 1	Worker	$\bar{X}_n$ (s)	s_{n-1} (s)	ME (s)	CI 95%
RDD (Row)	1	12.454	4.367	± 3.124	[9.330, 15.578]
RDD (Tuple)	1	11.335	4.610	± 3.298	[8.037, 14.633]
DF	1	2.983	4.716	± 3.373	[0.0, 6.357]
RDD (Row)	2	14.825	5.969	± 4.268	[10.557, 19.093]
RDD (Tuple)	2	10.846	4.420	± 3.162	[7.684, 14.008]
DF	2	3.324	5.768	± 4.126	[0.0, 7.450]
RDD (Row)	3	13.672	8.012	± 5.731	[7.941, 19.403]
RDD (Tuple)	3	11.645	4.661	± 3.334	[8.311, 14.979]
DF	3	4.022	6.837	± 4.890	[0.0, 8.912]

Analisi e Commento dei Risultati (Query 1):

- **Efficienza del Motore DataFrame:** L'approccio **DF** si dimostra nettamente il più efficiente, riducendo i tempi medi di esecuzione ($\bar{X}_n$) a circa un quarto rispetto alle metodologie basate su RDD. Questo divario evidenzia l'efficacia delle ottimizzazioni automatiche del *Catalyst Optimizer*.
- **Impatto delle Tuple su RDD:** Il paradigma **RDD (Tuple)** garantisce un risparmio sistematico di tempo rispetto a **RDD (Row)**. La conversione preventiva dei record in tuple native Python riduce l'overhead di lookup e di memoria associato agli oggetti strutturati di Spark.
- **Overhead di Parallelismo:** L'incremento dei nodi worker da 1 a 3 non produce una riduzione dei tempi, bensì un lieve degrado prestazionale (evidente nei DataFrame, che salgono da 2.98s a 4.02s). Tale comportamento indica che per la Query 1 il costo di coordinamento della rete, scheduling e shuffle supera il beneficio computazionale della scalabilità orizzontale.

TABLE V
RISULTATI QUERY 2

Query 2	Worker	$\bar{X}_n$ (s)	s_{n-1} (s)	ME (s)	CI 95%
RDD (Row)	1	19.299	4.593	± 3.286	[16.013, 22.585]
RDD (Tuple)	1	12.729	2.614	± 1.870	[10.859, 14.599]
DF	1	2.803	4.388	± 3.139	[0.0, 5.942]
RDD (Row)	2	22.592	6.885	± 4.925	[17.667, 27.517]
RDD (Tuple)	2	12.920	4.646	± 3.323	[9.597, 16.243]
DF	2	3.879	6.216	± 4.446	[0.0, 8.325]
RDD (Row)	3	22.079	6.945	± 4.968	[17.111, 27.047]
RDD (Tuple)	3	13.373	4.391	± 3.141	[10.232, 16.514]
DF	3	4.223	7.756	± 5.548	[0.0, 9.771]

Analisi e Commento dei Risultati (Query 2):

- **Dominio Netto dei DataFrame:** L'approccio **DF** si conferma la soluzione in assoluto più performante, con tempi medi a regime drasticamente inferiori (tra i 2.8s e i 4.2s) rispetto agli RDD. Questo ribadisce l'enorme

vantaggio competitivo fornito dalle ottimizzazioni logiche del *Catalyst Optimizer*.

- **Impatto Cruciale delle Tuple negli RDD:** A differenza della query precedente, qui il divario prestazionale interno al mondo RDD è molto più marcato. L'approccio **RDD (Tuple)** abbatta i tempi di quasi il 40% rispetto a **RDD (Row)** (es. da 22.5s a 12.9s con 2 worker). L'eliminazione dell'overhead di deserializzazione degli oggetti Row produce benefici evidenti su operazioni più complesse.
- **Inefficacia della Scalabilità Orizzontale:** L'aumento dei worker da 1 a 3 non genera alcuno *speed-up*. Al contrario, si registra un sistematico e lieve degrado delle prestazioni su tutti e tre gli approcci. Questo comportamento conferma che, per moli di dati o complessità computazionali limitate, il costo architetturale legato allo *shuffle* dei dati e al coordinamento di rete tra i nodi supera il vantaggio della parallelizzazione.

TABLE VI
RISULTATI QUERY 3

Query 3	Worker	$\bar{X}_n$ (s)	s_{n-1} (s)	ME (s)	CI 95%
RDD (Row)	1	42.642	4.225	± 3.023	[39.619, 45.665]
RDD (Tuple)	1	32.458	2.693	± 1.927	[30.531, 34.385]
DF	1	6.507	5.642	± 4.036	[2.471, 10.543]
RDD (Row)	2	43.475	8.406	± 6.014	[37.461, 49.489]
RDD (Tuple)	2	24.958	4.945	± 3.538	[21.420, 28.496]
DF	2	7.732	7.653	± 5.475	[2.257, 13.207]
RDD (Row)	3	34.507	9.610	± 6.874	[27.633, 41.381]
RDD (Tuple)	3	26.478	5.728	± 4.097	[22.381, 30.575]
DF	3	8.379	8.026	± 5.741	[2.638, 14.120]

Analisi e Commento dei Risultati (Query 3):

- **Efficienza e Dominio dei DataFrame:** Coerentemente con le query precedenti, l'approccio **DF** surclassa le metodologie basate su RDD, chiudendo le esecuzioni con tempi medi compresi tra i 6.5s e gli 8.3s. La maggiore complessità computazionale della Query 3 esalta l'efficacia del *Catalyst Optimizer*, che minimizza le letture e ottimizza il piano fisico di esecuzione.
- **Impatto della Serializzazione negli RDD:** Il divario tra i due approcci MapReduce è qui particolarmente marcato. L'utilizzo di **RDD (Tuple)** garantisce un abbattimento dei tempi fino a 18 secondi rispetto a **RDD (Row)** (visibile nella configurazione a 2 worker). Questo dimostra come, all'aumentare della mole di elaborazione, l'overhead di serializzazione/deserializzazione degli oggetti Row nativi in Python diventi un collo di bottiglia critico.
- **Limiti della Scalabilità Orizzontale:** L'incremento del numero di worker evidenzia dinamiche non lineari. Per l'approccio DF si osserva addirittura un progressivo degrado prestazionale (da 6.507s con 1 worker a 8.379s con 3 worker). Questo andamento, indica che i tempi di computazione pura sono talmente ridotti che il costo di *shuffle* dei dati, della rete e del *task scheduling* tra

nodi multipli finisce per dominare il tempo di esecuzione complessivo.

VI. CONCLUSIONI

L'analisi sperimentale condotta sulle diverse query ha permesso di valutare criticamente le prestazioni del framework Apache Spark al variare del paradigma di programmazione e delle risorse di calcolo allocate.

In primo luogo, i risultati confermano in modo inequivocabile la netta superiorità dell'API strutturata basata su **DataFrame**. L'astrazione dichiarativa di alto livello, unita alle ottimizzazioni trasparenti del *Catalyst Optimizer*, ha garantito tempi di esecuzione a regime drasticamente inferiori rispetto alle controparti RDD. Questo dimostra come la delega del piano di esecuzione logico e fisico direttamente al motore di Spark, rappresenti la scelta architetturale ottima per i carichi di lavoro analitici.

In secondo luogo, all'interno del paradigma **RDD**, l'esperimento ha evidenziato le criticità legate all'uso di PySpark. L'implementazione standard (basata su oggetti `Row`) soffre di un grave collo di bottiglia causato dalla continua serializzazione e deserializzazione dei dati tra la JVM e l'interprete Python. La conversione preventiva dei record in `Tuple` native si è rivelata una strategia di ottimizzazione cruciale, in grado di abbattere sistematicamente i tempi di esecuzione (particolarmente evidente nelle query più complesse) riducendo l'overhead di accesso ai dati.

Dal punto di vista della **scalabilità orizzontale**, l'incremento del numero di nodi worker da 1 a 3 non ha prodotto il miglioramento prestazionale teorico (*speed-up*). Al contrario, i tempi medi hanno mostrato un andamento altalenante o in lieve degrado. Questo fenomeno sottolinea che, per operazioni sufficientemente rapide o con moli di dati contenute, il costo computazionale legato allo *scheduling* dei task, al coordinamento di rete e alle fasi di *shuffle* supera ampiamente i benefici fisici della parallelizzazione. Risulta pertanto fondamentale dimensionare il cluster in modo strettamente proporzionale al reale volume dei dati per evitare colli di bottiglia architetturali.

Infine, l'analisi statistica dei tempi di esecuzione ha messo in luce il forte impatto del *warm-up* iniziale. Le elevate varianze registrate a causa dei picchi della prima iterazione confermano che la valutazione delle prestazioni in ambienti Big Data distribuiti basati su JVM richiede particolare cautela: il reale comportamento a regime del sistema può essere apprezzato solo dopo aver superato le latenze fisiologiche di inizializzazione del contesto e allocazione delle risorse.

REFERENCES

- [1] Types of Delay: https://www.aspm.faa.gov/aspmhelp/index/Types_of_Delay.html